Data and Algorithm Analysis

Chapter 30 — Polynomial and the FFT

Thang Hoang

Department of Computer Science, Virginia Tech

Outline

Overview

Polynomials

Polynomial Representations

The DFT and FFT

Table of Contents

Overview

Polynomials

Polynomial Representations

The DFT and FFT

Overview

Two ways to represent polynomials of degree n:

1. Coefficient representation. Multiplication takes $\Theta(n^2)$ time.
2. Point-value representation. Multiplication takes $\Theta(n)$ time.

Will see how to convert back and forth in $\Theta(n \lg n)$ time by using the fast Fourier transform (FFT), which will yield a $\Theta(n \lg n)$-time algorithm for multiplication in coefficient form.

Fourier Transforms

Commonly used in signal processing.

Signal is given in the **time domain** mapping time to amplitude.

Fourier analysis expresses the signal as a weighted sum of phase-shifted sinusoids of varying frequencies.

Weights and phases associated with the frequencies characterize the signal in the **frequency domain**.

FFT is a fast way to compute a Fourier transform.

Goal: Design a circuit to compute the FFT.

Table of Contents

Overview

Polynomials

Polynomial Representations

The DFT and FFT

Polynomial

A polynomial in the variable $x \in F$ over an algebraic field F represents a function $A(x)$ as a formal sum

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

where $a_0, a_1, \ldots, a_{n-1} \in F$ are polynomial coefficients.

Typically, $F = \mathbb{C}$ (complex numbers).

Polynomial A(x) has **degree** k ($\deg(A) = k$) if its highest non-zero coefficient is a_k.

Any integer strictly greater than the degree of a polynomial is a **degree-bound** of the polynomial.

Arithmetic Operations on Polynomials

Addition: If $A(x)$ and $B(x)$ are deg-n polynomials, their sum is a deg-n polynomial $C(x)$ such that $C(x) = A(x) + B(x)$ for all $x \in F$.

Let $A(x) = \sum_{j=0}^{n-1} a_j x^j$, $B(x) = \sum_{j=0}^{n-1} b_j x^j$, then

$$C(x) = \sum_{j=0}^{n-1} c_j x^j = \sum_{j=0}^{n-1} (a_j + b_j) x^j$$

Example

Let $A(x) = 6x^3 + 7x^2 - 10x + 9$, $B(x) = -2x^3 + 4x - 5$.
Then, $C(x) = A(x) + B(x) = 4x^3 + 7x^2 - 6x + 4$.

Arithmetic Operations on Polynomials

Multiplication: If $A(x)$ and $B(x)$ are deg-n polynomials, their **product** is a deg-$(2n-1)$ polynomial $C(x)$ such that $C(x) = A(x)B(x)$ for all $x \in F$.

Let $A(x) = \sum_{j=0}^{n-1} a_j x^j$, $B(x) = \sum_{j=0}^{n-1} b_j x^j$, then

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j = \sum_{j=0}^{2n-2} \left[\sum_{k=0}^{j} (a_k + b_{j-k}) \right] x^j$$

Example

Let $A(x) = 6x^3 + 7x^2 - 10x + 9$, $B(x) = -2x^3 + 4x - 5$.
Then, $C(x) = A(x)B(x) = \ldots$ (on board).

Table of Contents

Overview

Polynomials

Polynomial Representations

The DFT and FFT

Coefficient Representation

Coefficient representation of a polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$ is a vector of coefficients $\vec{a} = (a_0, a_1, \ldots, a_{n-1})$.

Evaluating the polynomial $A(x)$ at a given point x_0 consists of computing the value of $A(x_0)$ in Θn time using **Horner's rule**:

$$A(x) = a_0 + x_0 \left(a_1 + x_0 \left(a_2 + \cdots + x_0 \left(a_{n-1} + x_0 \left(a_{n-1} \right) \right) \ldots \right) \right)$$

Coefficient Representation

Let $\vec{a} = (a_0, a_1, \ldots, a_{n-1})$ and $\vec{b} = (b_0, b_1, \ldots, b_{n-1})$ be coefficient vectors of polynomials $A(x)$ and $B(x)$.

Polynomial addition takes $\Theta(n)$ time:

- Compute the coefficient vector $\vec{c} = (c_0, c_1, \ldots, c_n)$, where $c_j = a_j + b_j$ for $j = 0, 1, \ldots, n-1$.

Polynomial multiplication takes $\Theta(n^2)$ time:

- Compute the coefficient vector $\vec{c} = (c_0, c_1, \ldots, c_{2n-2})$, where $c_j = \sum_{k=0}^{j}(a_k b_{j-k})$ for $j = 0, 1, \ldots, 2n-2$.
- $\vec{c}$ is called the **convolution** of $\vec{a}$ and $\vec{b}$, i.e., $\vec{c} = \vec{a} \otimes \vec{b}$.

Point-Value Representation

A **point-value representation** of a deg-n polynomial $A(x)$ is a set of n *point-value pairs*

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$$

such that

- All $x_0, x_1, \ldots, x_n$ are distinct.
- $y_k = A(x_k)$ for $k = 0, 1, \ldots, n-1$.

Point-Value Representation

A polynomial has many different point-value representations.

Any set of n distinct points serves as a **basis** for the representation.

To compute a point-value representation, select n distinct points $x_0, x_1, \ldots, x_{n-1}$ and then evaluate $A(x_k)$ for $k = 0, 1, \ldots, n-1$.

- With Horner's method, evaluating at n points takes $\Theta(n^2)$ time.
- By choosing appropriate x_k, this can run in $\Theta(n \lg n)$ time.

Determining the coefficient form of a polynomial from a point-value representation is called **interpolation**.

Interpolation is well defined when the polynomial has a degree-bound *equal* to the given number of point-value pairs.

Point-Value Representation

Theorem (Uniqueness of an interpolating polynomial)

For any set $\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$ *of n point-value pairs such that all the* x_k *values are distinct, there is a unique deg-n polynomial* $A(x)$ *such that* $y_k = A(x_k)$ *for* $k = 0, 1, \ldots, n-1$.

Proof.

On board □

Point-Value Representation

Let $\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$ and $\{(x_0, y'_0), (x_1, y'_1), \ldots, (x_{n-1}, y'_{n-1})\}$ be the point-value representations for $A(x)$ and $B(x)$.

Addition: A point-value representation for $C(x) = A(x) + B(x)$ is

$$\{(x_0, y_0 + y'_0), (x_1, y_1 + y'_1), \ldots, (x_{n-1}, y_{n-1} + y'_{n-1})\}$$

Addition of two polynomials in point-value representation takes $\Theta(n)$ time.

Point-Value Representation

Multiplication: Given $A(x), B(x)$, and their point-value representation, compute $C(x) = A(x)B(x)$.

Observation: If $C(x) = A(x)B(x)$, $C(x_k) = A(x_k)B(x_k)$ for any x_k.

- Point-wise multiply point-value representations of A and B to obtain point-value presentation for C.

Recall: In polynomial multiplication, $\deg(C) = \deg(A) + \deg(B)$.

- If A and B have degree-bound n, C has degree-bound $2n$.

Point-value representation for A and B consists of n point-value pairs
$\Rightarrow$ Point-wise multiplication only gives n point-value pairs.

- Need $2n$ pairs to interpolate C. ***How?***

Point-Value Representation

Extend point-value representations for A and B consisting of $2n$ point-value pairs each.

Let $\{(x_0, y_0), (x_1, y_1), \ldots, (x_{2n-1}, y_{2n-1})\}$ and $\{(x_0, y'_0), (x_1, y'_1), \ldots, (x_{2n-1}, y'_{2n-1})\}$ be the extended point-value representations for A and B. The a point-value representation for C is

$$\{(x_0, y_0 y'_0), (x_1, y_1 y'_1), \ldots, (x_{2n-1}, y_{2n-1} y'_{2n-1})\}$$

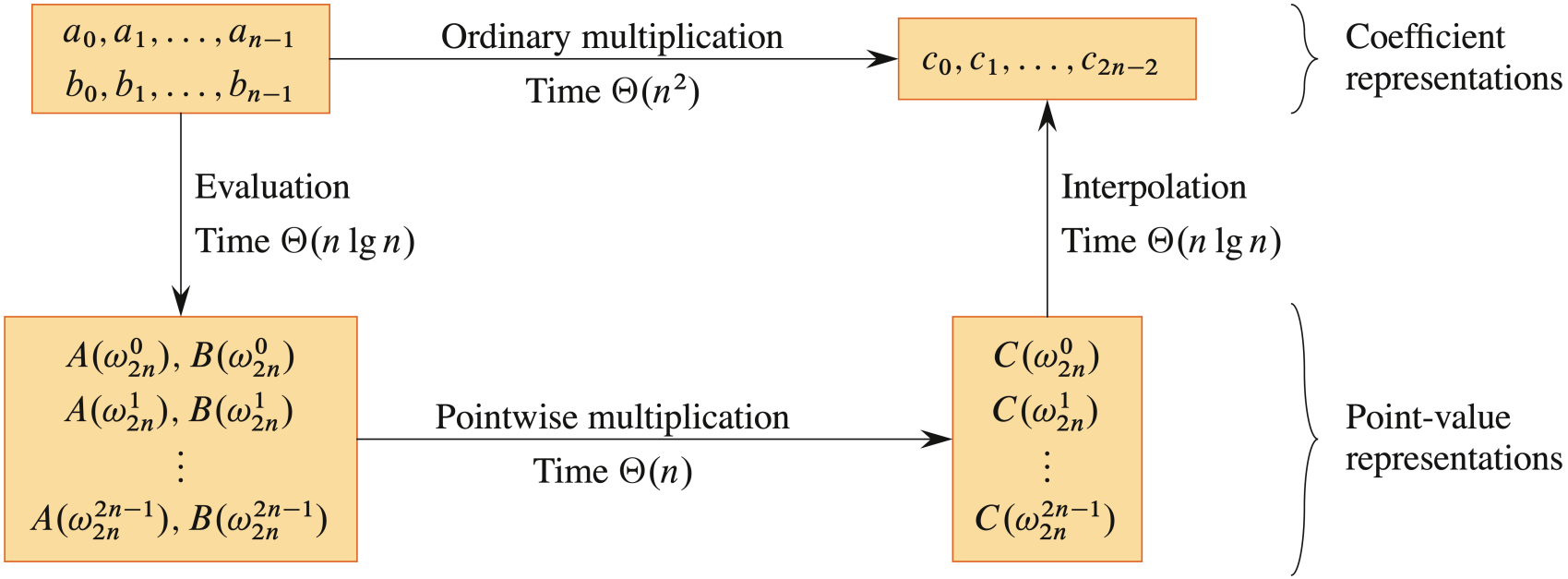
Multiplying polynomials in point-value form takes just $\Theta(n)$ time.

Question: How to extend point-value representations from n to $2n$ point-value pairs?

Fast Polynomial Multiplication in Coefficient Form

- Exploit linear-time multiplication in *point-value* form to expedite polynomial multiplication in *coefficient* form.
- Need to evaluate and interpolate quickly.
- Any points can be evaluation points $\Rightarrow$ choose special points that permit *fast* representation conversion (e.g., in $\Theta(n \lg n)$ time).
- **Complex roots of unity** as the evaluation points. Use FFT to evaluate and interpolate.

Fast Polynomial Multiplication in Coefficient Form



Fast Polynomial Multiplication in Coefficient Form

Four-step process:

1. **Double degree-bound:** Create coefficient representations of $A(x)$ and $B(x)$ as deg-$2n$ polynomials by adding n high-order zero coefficients to each. Time: $\Theta(n)$.
2. **Evaluate**: Compute point-value representations of $A(x)$ and $B(x)$ of length $2n$ with FFT of order $2n$ on each polynomial. These representations contain the values of the two polynomials at the $(2n)$th roots of unity. Time: $\Theta(n \lg n)$.

Fast Polynomial Multiplication in Coefficient Form

Four-step process:

3. **Point-wise multiply**: Compute a point-value representation for the polynomial $C(x) = A(x)B(x)$ by multiplying these values together point-wise. This representation contains the value of $C(x)$ at each $(2n)$th root of unity. Time: $\Theta(n)$.
4. **Interpolate**: Create the coefficient representation of the polynomial C(x) with FFT on $2n$ point-value pairs to compute the **inverse DFT**. Time: $\Theta(n \lg n)$.

Total time: ?

Theorem

It is possible to multiply two deg-n polynomials in $\Theta(n \lg n)$ time with both the input and output representations in coefficient form.

Table of Contents

Overview

Polynomials

Polynomial Representations

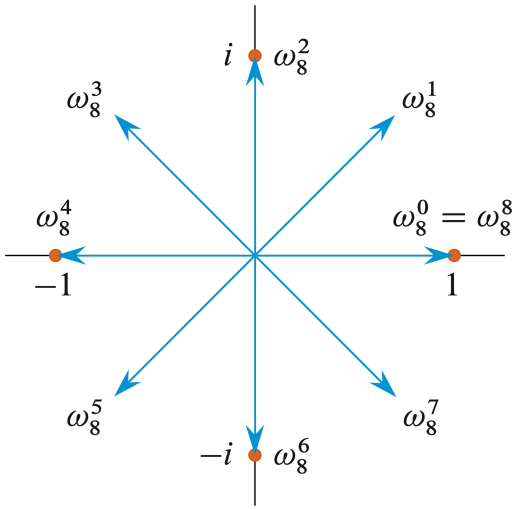
The DFT and FFT

Complex Roots of Unity

A complex nth root of unity is a complex number ω such that $\omega^n = 1$.

- There are exactly n complex nth roots of unity: $e^{2\pi ik/n}$ for $k = 0, 1, \ldots, n-1$
- The **principal nth root of unity** is $\omega_n = e^{2\pi i/n}$.
- All other complex nth roots of unity are powers of ω_n.

Example



Complex Roots of Unity

The n complex nth roots of unity, $\omega_n^0, \omega_n^1, \ldots, \omega_n^{n-1}$ form a group under multiplication. This group has the same structure as the additive group $(\mathbb{Z}_n, +)$ modulo n

- $\omega_n^n = \omega_n^0 = 1$ implies that $\omega_n^j \omega_n^k = \omega_n^{(j+k) \mod n}$.
- Similarly, $\omega_n^{-1} = \omega_n^{n-1}$.

Lemma (Cancellation Lemma)

For any integers $n \geq 0, k \geq 0$, and $d > 0$, $\omega_{dn}^{dk} = \omega_n^k$.

Corollary

For any even integer $n > 0$, $\omega_n^{n/2} = \omega_2 = -1$.

Complex Roots of Unity

Lemma (Halving Lemma)

If $n > 0$ is even, then the squares of the n complex nth roots of unity are the $(n/2)$ complex $(n/2)$th roots of unity.

Lemma (Summation Lemma)

For any integer $n \geq 1$ and nonzero integer k not divisible by n, $\sum_{j=0}^{n-1} (\omega_n^k)^j = 0$.

The DFT

DFT($\vec{a}$)

Input: coefficient vector $\vec{a} = (a_0, a_1, \ldots, a_{n-1})$ of a polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$

Output: $y_1, \ldots, y_2, \ldots, y_{n-1}$, where $y_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j \omega_n^{kj}$ (Evaluations of A at n complex nth root of unity)

The FFT

Fast Fourier transform (FFT) uses properties of the complex roots of unity to compute $\mathrm{DFT}_n(\vec{a})$ in $\Theta(n \lg n)$ time.

Divide-and conquer strategy: Separate the <u>even</u>-indexed and <u>odd</u>-indexed coefficients of $A(x)$ to define the two deg-$(n/2)$ new polynomials $A^{\text{even}}(x)$ and $A^{\text{odd}}(x)$ of degree-bound $n/2$:

- $A^{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \cdots + a_{n-2} x^{n/2-1}$.
- $A^{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \cdots + a_{n-1} x^{n/2-1}$.

$$A(x) = A^{\text{even}}(x^2) + xA^{\text{odd}}(x^2)$$

The FFT

Evaluating $A(x)$ at $\omega_n^0, \omega_n^1, \ldots, \omega_n^{n-1}$ becomes

- Evaluate the deg-$(n/2)$ polynomials $A^{\text{even}}(x^2)$ and $A^{\text{odd}}(x^2)$ at points $(\omega_n^0)^2, (\omega_n^1)^2, \ldots, (\omega_n^{n-1})^2$.
- Combine the results as $A(x) = A^{\text{even}}(x^2) + xA^{\text{odd}}(x^2)$.

FFT Procedure

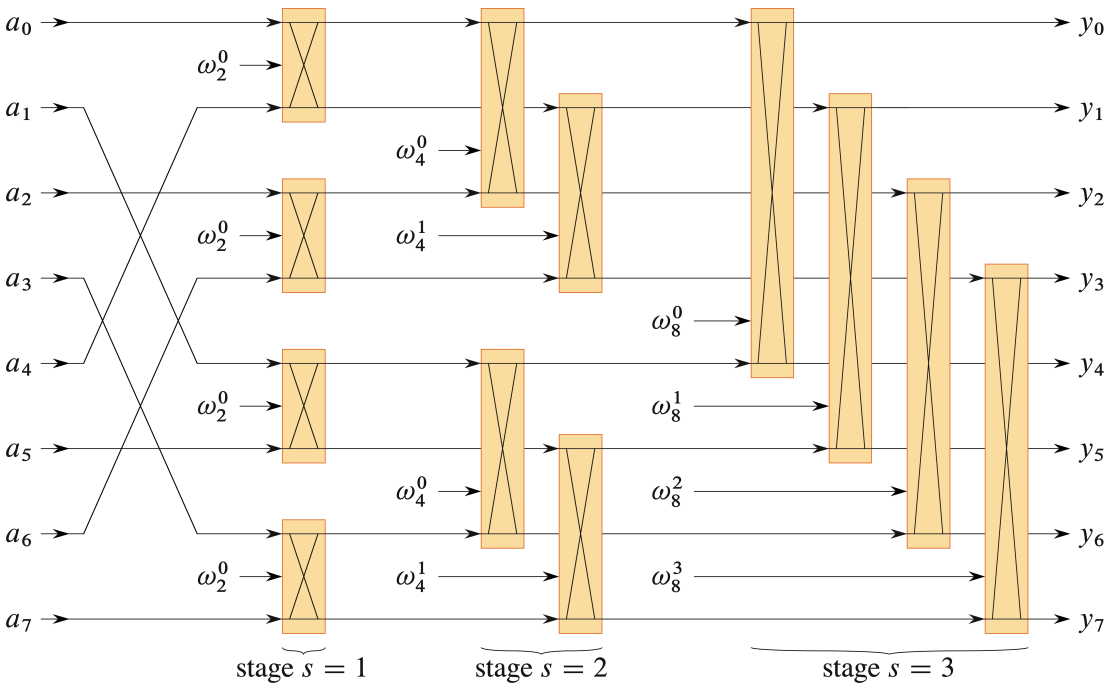
By Halving Lemma, the list $(\omega_n^0)^2, (\omega_n^1)^2, \ldots, (\omega_n^{n-1})^2$ consists **not** of n distinct values but only of $(n/2)$ complex $(n/2)$th roots of unity, with each root occurring exactly twice.

FFT recursively evaluates deg-$(n/2)$ A^{even} and A^{odd} at $n/2$ complex $(n/2)$th roots of unity.

- Subproblems have the same form as the original problem, but are half the size.
- Divide an n-element DFT_n computation into two $(n/2)$-element $\text{DFT}_{n/2}$ computations.

FFT Pseudocode

```
if n = 1
    return a
ω_n = e^{2πi/n}
ω = 1
a^even = (a_0, a_2, ..., a_{n-2})
a^odd = (a_1, a_3, ..., a_{n-1})
y^even = FFT(a^even, n/2)
y^odd = FFT(a^odd, n/2)
for k = 0 to n/2 − 1
    y_k = y_k^even + ω y_k^odd
    y_{k+(n/2)} = y_k^even − ω y_k^odd
    ω = ω ω_n
return y
```



Running Time: ?

Interpolation at the Complex Roots of Unity

For polynomial multiplication, need to convert from coefficient form to point-value form.

Evaluating the polynomial at the complex roots of unity $\Rightarrow$ pointwise multiply $\Rightarrow$ convert from point-value back to coefficient by interpolating.

How to interpolate?

Interpolation at the Complex Roots of Unity

Write the DFT as a matrix equation $\vec{y} = V_n \vec{a}$, where V_n is a Vandermonde matrix containing the appropriate powers of ω_n

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \cdots & 1 & \\ 1 & \omega_n & \omega_n^2 & \omega_n^3 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \omega_n^6 & \cdots & \omega_n^{2(n-1)} \\ 1 & \omega_n^3 & \omega_n^6 & \omega_n^9 & \cdots & \omega_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \omega_n^{3(n-1)} & \cdots & \omega_n^{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix}$$

For inversion, $\vec{a} = \mathrm{DFT}_n^{-1}(\vec{y})$, computes $\vec{a} = V_n^{-1}\vec{y}$.

Interpolation at the Complex Roots of Unity

Theorem

For $j, k = 0, 1, \ldots, n-1$, the (j, k) entry of V_n^{-1} is $\omega_n^{-jk/n}$.

Proof.

Just need to show that $V_n^{-1} V_n = I$ ($n \times n$ identity matrix). □

FFT Circuits

Because FFT's applications in signal processing require fast speed, often implemented as a circuit in hardware.

FFT's divide-and-conquer structure allows for a circuit with a parallel structure.

The circuit **depth** (maximum number of computational elements between an output and any input that can reach it) is $\Theta(n)$.

FFT Butterfly Operations

Recall the following lines in the **for** loop of FFT procedure:

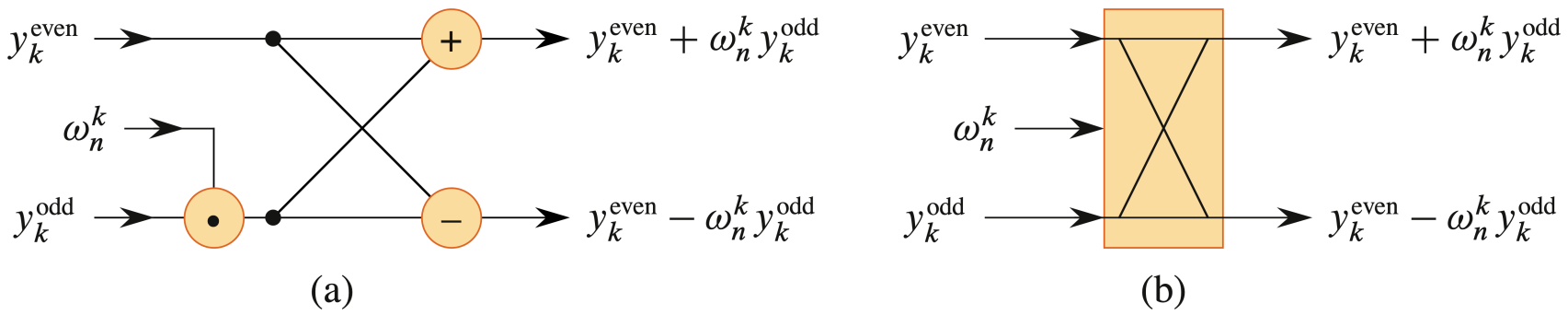
```
for k = 0 to n/2 − 1
    y_k = y_k^even + ω y_k^odd
    y_{k+(n/2)} = y_k^even − ω y_k^odd
    ω = ω ω_n
return y
```

ωy_k^{odd} is computed twice per iterationx.
Optimize the loop by pulluing out **common subexpression**.

$$t = \omega y_k^{\text{odd}}$$
$$y_k = y_k^{\text{even}} + t$$
$$y_{k+(n/2)} = y_k^{\text{even}} - t$$

This is called **butterfly operation**.

FFT Butterfly Operations



FFT Recursive Circuit Structure

Recall that FFT procedure follows divide-and-conquer strategy:

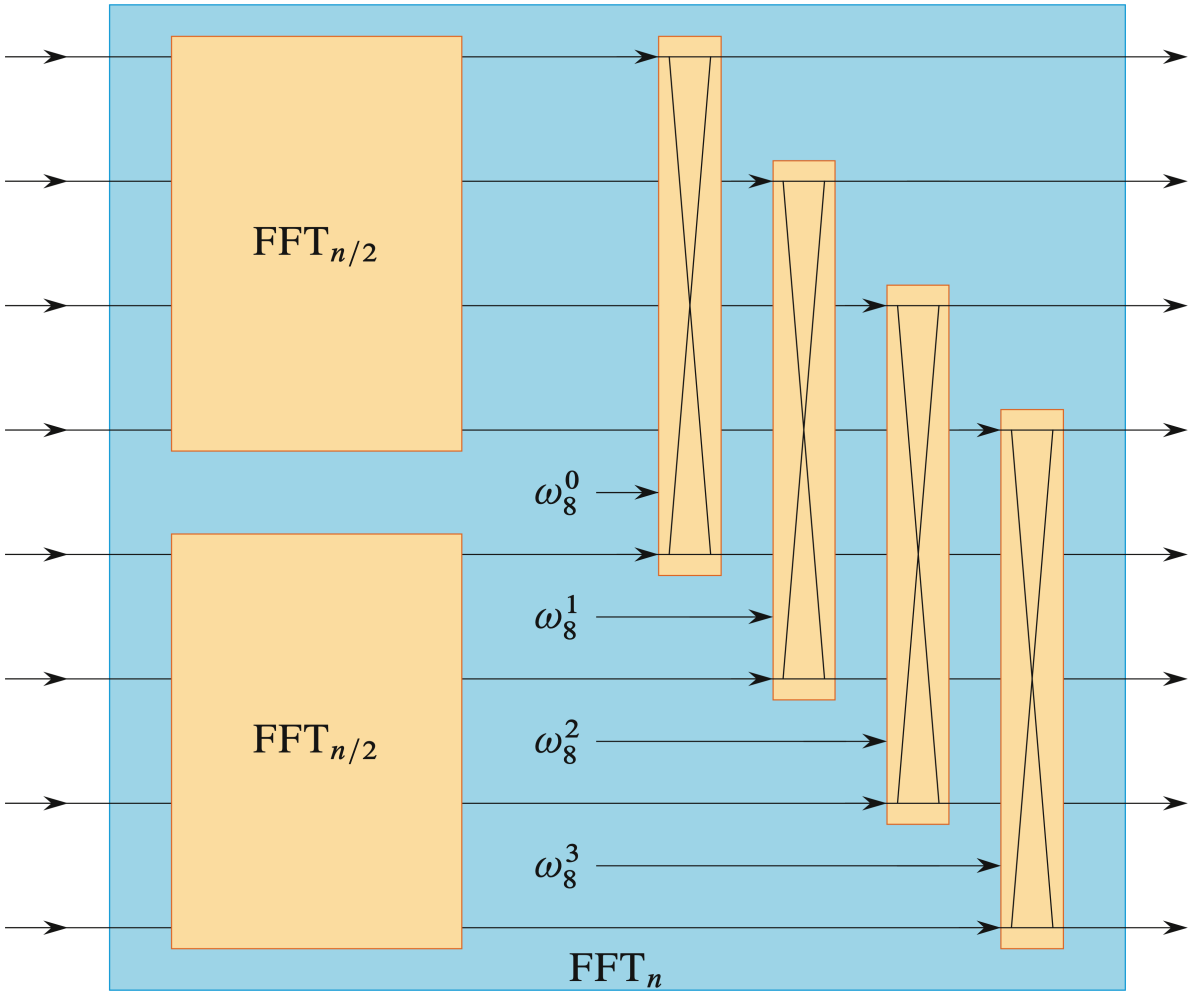
Divide the n-element input vector into its $n/2$ even-indexed and $n/2$ odd-indexed elements.

Conquer by recursively computing the DFTs of the two subproblems, each of size $n/2$.

Combine by performing $n/2$ butterfly operations.

- These butterfly operations work with twiddle factors $\omega_n^0, \omega_n^1, \ldots, \omega_n^{n/2-1}$.

FFT Recursive Circuit Structure



Permuting the Inputs

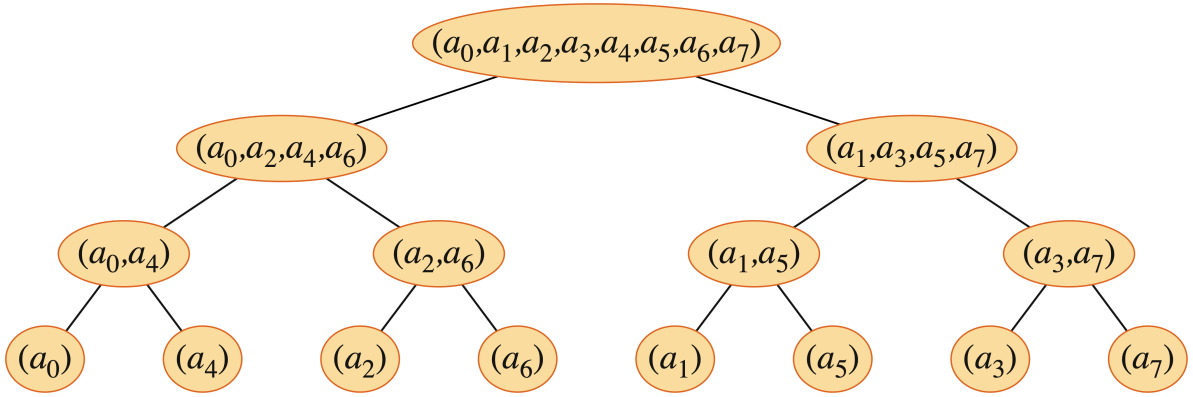
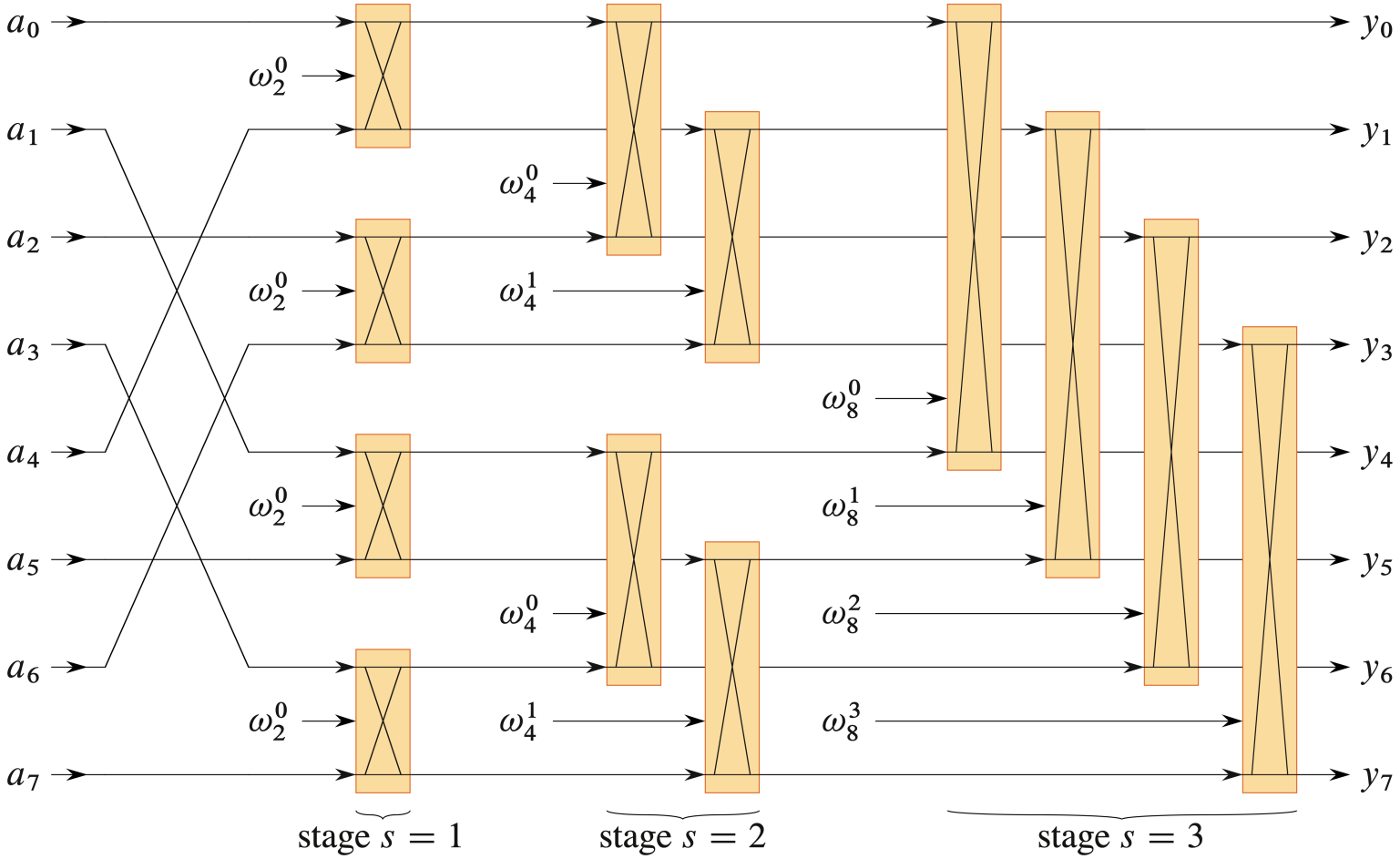
How does the circuit incorporate the divide step?

Idea: Understand how input vectors to the various recursive calls of the FFT relate to the original input vector to emulate the divide step at the start for all recursion levels.

- Bit-reversal permutation.

Example

Tree of input vectors to the recursive calls of FFT for $n = 8$:



The Full FFT Circuit